Практическая работа №2

Аудит готового клиент-серверного приложения

ФИО: Богданов Роман Романович
Группа: ИС41

Цель работы - проверить обмен данными между клиентом и сервером, исследовать ошибки и сравнить JSON и XML. В обычном режиме оба протокола вернули OK. При проверке форматов обнаружена неполная проверка восстановления XML.

1 Ответы на вопросы

Вопрос 1 Протокол функции send_reliable

Используется TCP. В функции создаётся сокет SOCK_STREAM и вызывается connect(), то есть сначала устанавливается соединение с сервером.

Вопрос 2 Протокол функции send_fast

Используется UDP. Тип SOCK_DGRAM означает передачу отдельных датаграмм, а sendto() отправляет сообщение по указанному адресу без предварительного соединения.

Вопрос 3 Формат передаваемых данных

Клиент передаёт JSON, закодированный в UTF-8. Это подтверждает строка отправки:

```
client.send(json.dumps(message,
    ensure_ascii=False).encode("utf-8"))
```

Вопрос 4 Отправка обычной строки hello

Байты b"hello" не являются корректным JSON. Сервер перехватывает JSONDecodeError, выводит сообщение об ошибке и возвращает {"status": "ERROR", "reason": "invalid JSON"}.

2 Запуск сервера и клиента

Сервер и клиент запущены в двух отдельных терминалах VS Code в Windows. Сервер принимает TCP на порту 9000 и UDP на порту 9001. На скриншоте показаны запуск программ и успешный обмен данными.

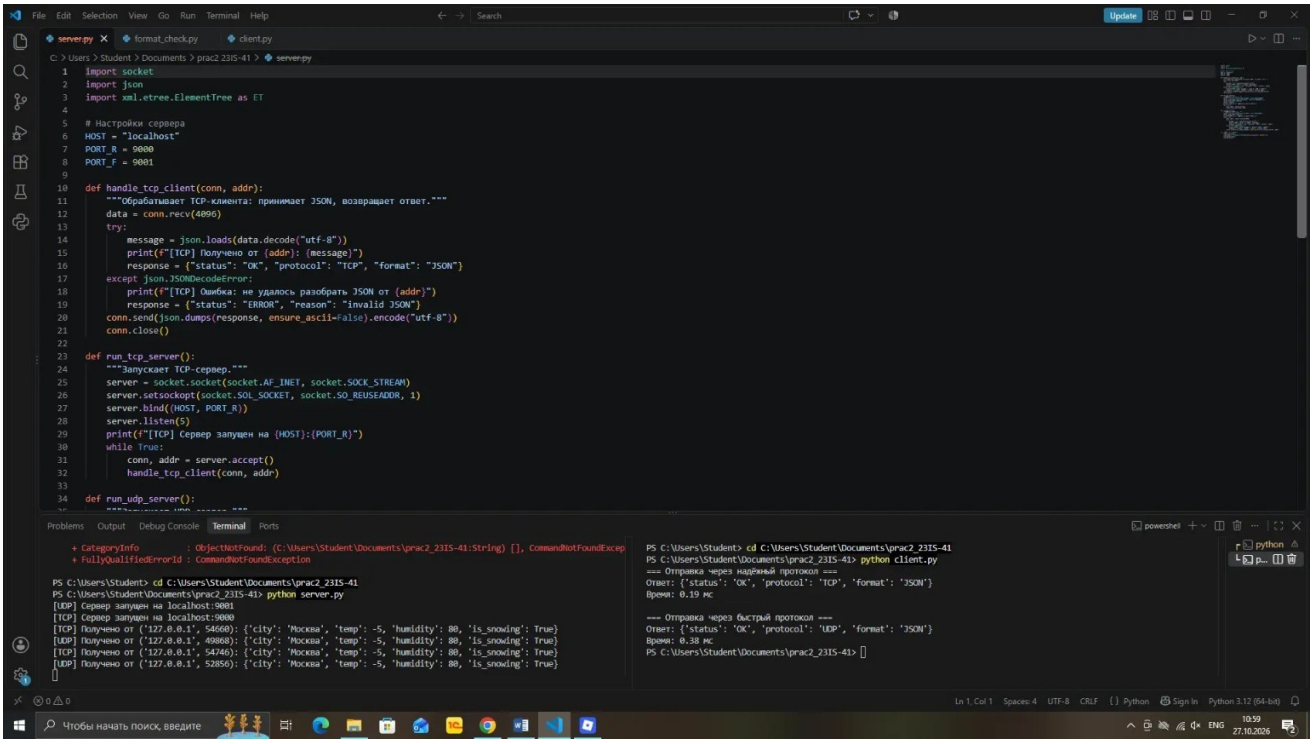


Рисунок 1 - Сервер и клиент в VS Code. Слева работает server.py, справа выполнен client.py.

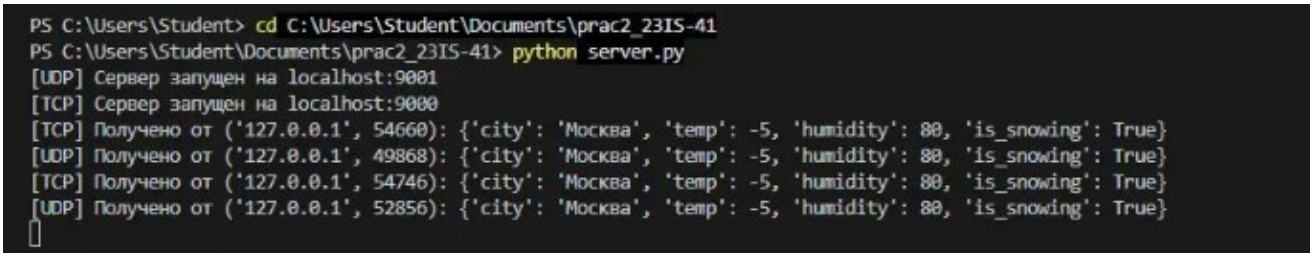


Рисунок 2 - Терминал сервера крупнее. Получены данные о погоде по TCP и UDP.

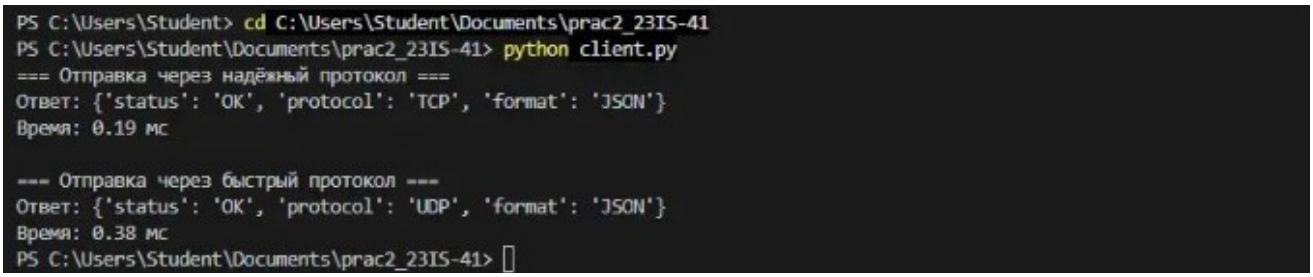


Рисунок 3 - Терминал клиента крупнее. Для TCP и UDP получен статус OK. В этом запуске время составило 0,19 мс и 0,38 мс соответственно. Одного запуска недостаточно для сравнения скорости протоколов; установка TCP-соединения в измерение не входит.

3 Тестирование

Граничные случаи дополнительно проверены в Linux. Поэтому ошибка отключённого TCP-сервера ниже показана в формате Linux; в Windows её номер и текст могут отличаться.

Случай 1 Сервер выключен и запущен TCP-клиент

```
ConnectionRefusedError: [Errno 111] Connection refused
```

Подключение отклонено, потому что порт 9000 никто не слушает. Ошибка не обработана в клиенте, поэтому программа завершается и до UDP-части не доходит. В Windows текст и номер ошибки могут отличаться.

Случай 2 Сервер выключен и запущен только UDP-клиент

```
{'status': 'TIMEOUT'}
Ожидание: 2.002 с
```

Для отдельной проверки вызвана функция send_fast(weather), без TCP-части. Датаграмма отправлена, но ответа нет. Через примерно 2 секунды сработал установленный таймаут, и функция вернула TIMEOUT.

Случай 3 Отправлено hello вместо JSON

```
{"status": "ERROR", "reason": "invalid JSON"}
```

Отдельный проверочный клиент отправил b"hello" через TCP. В журнале сервера появилось «Ошибка: не удалось разобрать JSON». Сервер вернул ERROR и закрыл соединение; исходный client.py оставлен без изменений.

Случай 4 Сравнение JSON и XML

```
Размер JSON: 72 байт
Размер XML: 111 байт
Данные совпадают: True
```

В этом наборе данных JSON компактнее на 39 байт. XML длиннее из-за открывающих и закрывающих тегов. Размеры измерены в UTF-8, с тем оформлением, которое создаёт исходная программа.

JSON восстановил число -5, число 80 и логическое значение True. При восстановлении XML получились строки "-5", "80" и "True". Строка «Данные совпадают: True» относится только к restored_json == weather и ничего не доказывает о совпадении XML.

4 Вывод

Для этого приложения я выбрал бы TCP и JSON. Здесь важно получить подтверждение от сервера, а TCP обеспечивает упорядоченный поток байтов и повторную передачу при потерях, пока соединение работает. JSON удобен для небольшого объекта погоды: он читаемый, компактнее полученного XML и сохраняет используемые типы данных.

UDP подошёл бы для частых обновлений показаний, когда потеря отдельного сообщения допустима и скоро придёт новое. Если каждое сообщение обязательно должно быть обработано, одного UDP без дополнительных механизмов недостаточно.

Что стоит исправить в программе

1. Обработать ошибку подключения TCP и установить таймаут. Сейчас при выключенном сервере клиент аварийно завершается.

2. Использовать sendall() и чтение полного сообщения. Один send() не гарантирует отправку всех байтов, а один recv(4096) - получение всего JSON. Для TCP нужно договориться о границах сообщений, например передавать длину перед данными.

3. Преобразовывать типы при чтении XML и отдельно сравнивать restored_xml с weather. Сейчас результат сравнения выводится только для JSON.

4. Установить таймаут чтения на сервере. TCP-клиенты обслуживаются по очереди, поэтому подключившийся клиент, который ничего не отправляет, может задержать обработку остальных.

5. Добавить обработку ошибок UTF-8 и сетевых операций. Сейчас обработчик перехватывает только ошибку разбора JSON.

Итог: обычный обмен и обработка некорректного JSON работают. Основные недостатки связаны с обработкой сетевых сбоев, чтением TCP-сообщений и проверкой восстановления XML.